INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

MNA - Maestría Inteligencia Artificial Aplicada

Avance 2. Ingeniería de características

Jeanette Ríos Martínez **A01688888**

Ali Mateo Campos Martínez **A01796071**

Ali Alfonso Rico Vázquez **A01350630**

Docente Titular:

Dra. Grettel Barceló Alonso

Dr. Luis E. Falcón Morales

Docente Asistente:

Mtra. Verónica Sandra Guzmán de Valle

Asesor de proyecto:

Dr. Guillermo Mota Medina

Sports Bet Engine

17 Mayo 2026

Ingeniería de características (NBA - JUGADORES) **3**
Ingeniería de características (FÚTBOL - EQUIPOS) **5**
Selección de características **9**
Ingeniería de características para datos no estructurados **10**
Conclusiones de la fase **17**
Referencias **18**

Ingeniería de características (NBA - JUGADORES)

Jugador: Nikola Jokic

Equipo: Denver

¿Ha cambiado de equipo? No

Temporadas: desde 2015-16 hasta 2025-26

Crear nuevos datos (data frame del jugador):

Vamos a crear nuevos datos a partir de los que ya tenemos, es decir, sin consultar nuevos endpoints de la API.

Podemos obtener:

- ¿El equipo jugó de local o visitante?
- ¿Cuántos días de descanso hubo entre juegos?
- ¿Cuál es la abreviación del nombre del equipo oponente?
- ¿Cuál es el ID del equipo oponente?

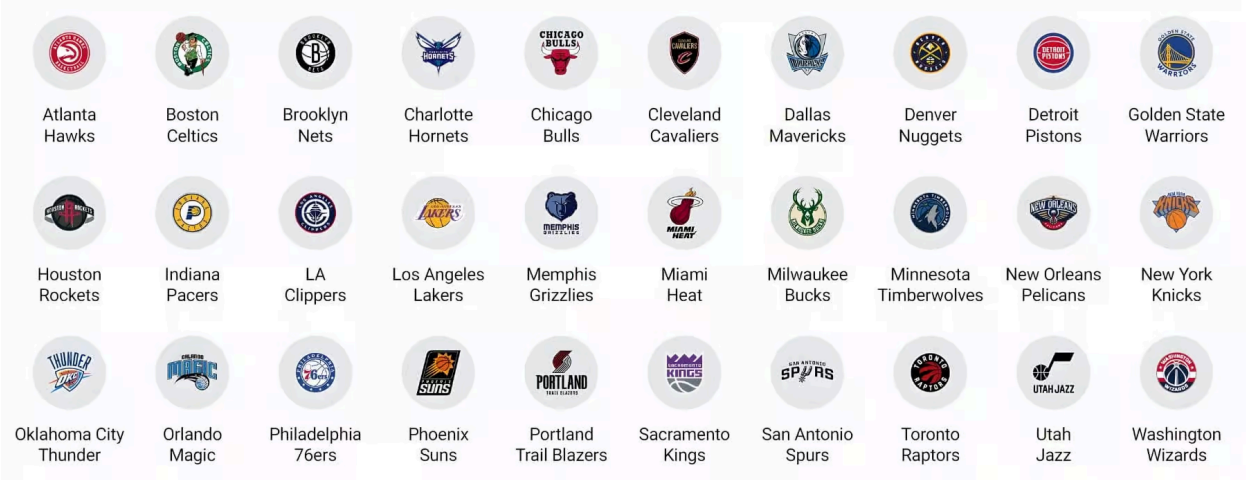
```python
# Nos aseguramos que el tipo de dato sea str
df_train['MATCHUP'] = df_train['MATCHUP'].astype(str)

# ¿Juegan de local o de visitante?
df_train["IS_HOME"] = df_train["MATCHUP"].apply(lambda x: 1 if "@" not in str(x) else 0)

# Establecer como tipo fecha
df_train["GAME_DATE"] = pd.to_datetime(df_train["GAME_DATE"])

# Ordenar el df en base a la fecha del juego
df_train = df_train.sort_values(by="GAME_DATE")

# Obtenemos los días de descanso entre juegos
df_train["DAYS_REST"] = df_train["GAME_DATE"].diff().dt.days

# Llenamos los días faltantes con 0 días de descanso
df_train["DAYS_REST"] = df_train["DAYS_REST"].fillna(0)

# Obtenemos el nombre del oponente
df_train['OPPONENT'] = df_train['MATCHUP'].apply(lambda x: x.split(' ')[-1])

display(df_train.head(5))
display(df_train.shape)
```

MIN_SEC	TEAM_COUNT	IS_HOME	DAYS_REST	OPPONENT	OPPONENT_TEAM_ID
23:02	1	0	0.0	LAC	1610612746
18:17	1	0	4.0	DAL	1610612742
19:55	1	1	2.0	CHI	1610612741

Obtener estadísticas de los oponentes:

Nuestro objetivo es obtener las estadísticas generales de todos los partidos jugados por los oponentes durante las temporadas analizadas, con el objetivo de enriquecer el dataframe del jugador y poder observar qué tan bien jugó el contrincante y cómo se comporta el jugador con ciertos equipos.

Actualmente la NBA cuenta con 30 equipos o franquicias, el obtener todos los partidos jugados desde la temporada 2015 hasta la temporada 2026 fue un proyecto de paciencia, sobre todo porque estamos usando una API gratuita, la cual limita el número de peticiones que podemos hacer por minuto.



Ingeniería de características (FÚTBOL - EQUIPOS)

¿Qué es un sistema de calificación ELO? [1]

El sistema de calificación ELO es un método para calcular los niveles de habilidad relativos de los equipos o jugadores individuales en juegos de suma cero.

A diferencia de las clasificaciones subjetivas de poder del fútbol, las calificaciones de ELO son puramente matemáticas: se actualizan después de cada partido en función de:

- quién ganó
- quién se esperaba que ganara
- cuánto ganaron

La idea central es simple. Si un equipo más fuerte vence a un equipo más débil, no cambia mucho. ¿Pero si es al revés? Ahí es cuando las calificaciones cambian drásticamente.
Esto hace que las calificaciones de Elo respondan increíblemente al rendimiento real en lugar de a la reputación o a los valores de mercado.

Nuestro objetivo:

Debido a la pequeña cantidad de datos (67 partidos al máximo nivel si es que llegas a todas las finales, como el PSG en la temporada 2024-2025), optamos por utilizar las variables de nuestro conjunto de datos en un sistema que nos permita saber si un equipo es fuerte o débil, es decir, un sistema ELO.

Con esto, buscamos entrenar un modelo “Poisson Regresor” en base a los goles anotados según el ELO de los equipos, y con esto poder hacer una predicción de los goles esperados.

CONSTRUCCIÓN DEL MODELO ELO

Inicialización:

Primero inicializamos todos los equipos con un puntaje estándar, en este caso elegimos el clásico 1,500 puntos.

```
teams = pd.concat([df['home_team'], df['away_team']]).unique()
elo_dictionary = {team: 1500 for team in teams}
elo_dictionary
```

```
{'Brentford': 1500,
 'Manchester United': 1500,
 'Watford': 1500,
 'Burnley': 1500,
 'Everton': 1500,
 'Leicester': 1500,
 'Chelsea': 1500,
 'Norwich': 1500,
 'Newcastle': 1500,
 'Tottenham': 1500,
 'Liverpool': 1500,
 'Aston Villa': 1500,
 'Leeds': 1500,
 'Manchester City': 1500,
 'Crystal Palace': 1500,
 'Brighton': 1500,
 'Wolves': 1500,
 'Southampton': 1500,
 'Arsenal': 1500,
 'West Ham': 1500,
 'Fulham': 1500,
 'Bournemouth': 1500,
 'Nottingham Forest': 1500,
 'Sheffield Utd': 1500,
 'Luton': 1500,
 'Ipswich': 1500,
 'Sunderland': 1500}
```

Reinicio cada temporada:

Cada nueva temporada tenemos que hacer un reinicio parcial de los puntos obtenidos, esto es debido al caos que se vive cada nueva temporada, los equipos se refuerzan o deshacen, cambios de DT, compra y venta de jugadores, personal médico, etc.

Actualmente nos quedamos con el 0.33% de los puntos obtenidos en la campaña.

```
def reboot_elo_new_seasson(elo_dictionary):

  for team, elo in elo_dictionary.items():
    elo_dictionary[team] = (elo * 0.67) + (1500 * 0.33)
```

Generación del ranking ELO:

```python
# HFA = ventaja de local (Home Field Advantage)
# K = Es un valor dinámico (respecto al torneo) para darle variedad a la puntuación
K, HFA = 20, 53
test_reboot_elo_ranking = True

# Listas temporales para guardar el estado ANTES del partido
elo_home_pre = []
elo_away_pre = []

current_year = int(AVAILABLE_SEASSONS[0])

# Recorremos todos los partidos
for index, row in df.iterrows():

  if row['season'] != current_year and test_reboot_elo_ranking:
    display(f"Last year: {current_year} // Current year {row['season']}")
    current_year = row['season']
    reboot_elo_new_seasson(elo_dictionary)

  home = row['home_team']
  away = row['away_team']
  goals_h = row['home_goals']
  goals_a = row['away_goals']

  # 1. Leer el elo actual de los equipos
  elo_current_h = elo_dictionary[home]
  elo_current_a = elo_dictionary[away]


  if index == 0:
    display(home)
    display(away)
    display(f"H: {elo_current_h} // A: {elo_current_a}")

  # 2. Guardarlo para el modelo predictivo
  elo_home_pre.append(elo_current_h)
  elo_away_pre.append(elo_current_a)

  # 3. Calcular la probabilidad con la fórmula de elo

# Agregamos +100 al local para simular la ventaja de jugar en casa (Home Field Advantage)
  E_home = 1 / (1 + 10 ** (((elo_current_a) - (elo_current_h + HFA)) / 400))
  E_away = 1 - E_home

  # 4. Determinar el resultado real
  # (1 = gana local, 0.5 = empate, 0 = gana visita)
  if goals_h > goals_a:
      S_home, S_away = 1, 0
  elif goals_h == goals_a:
      S_home, S_away = 0.5, 0.5
  else:
      S_home, S_away = 0, 1

  # 5. Multiplicar por el margen de victoria
  # Un 4-0 demuestra más superioridad que un 1-0
  # Usamos la raíz cuadrada de la diferencia para amplificar un resultado grande
  margin_goals = abs(goals_h - goals_a)
  margin_multiplier = math.sqrt(margin_goals) if margin_goals > 0 else 1.0

  # 6. Calcular los puntos ganados en el partido
  change_elo_home = K * margin_multiplier * (S_home - E_home)
  change_elo_away = K * margin_multiplier * (S_away - E_away)

  # 7. Actualizar el diccionario de los equipos con los elo
  elo_dictionary[home] = elo_current_h + change_elo_home
  elo_dictionary[away] = elo_current_a + change_elo_away

  # ¿No se debería ir actualizando el ELO respecto a cada partido?
  # 8. Añadir el elo al dataframe
  df.at[index, 'Elo_Home_Pre'] = elo_current_h
  df.at[index, 'Elo_Away_Pre'] = elo_current_a
  df.at[index, 'Elo_Diff'] = (elo_current_h - elo_current_a)
```

Observación del ranking final:

A continuación, podemos ver los resultados del ranking ELO desde la temporada 2021 hasta la temporada 2026. Es bastante acertado considerando que el Arsenal y el Manchester City han estado dominando la liga inglesa en los años analizados.

Arsenal	1715.08
Manchester City	1709.64
Liverpool	1608.49
Manchester United	1599.74
Bournemouth	1580.45
Brighton	1569.31
Aston Villa	1545.90
Nottingham Forest	1538.82
Brentford	1538.54
Everton	1529.46
Chelsea	1528.75
Newcastle	1526.09
Leeds	1511.91
Fulham	1504.56
Crystal Palace	1494.33

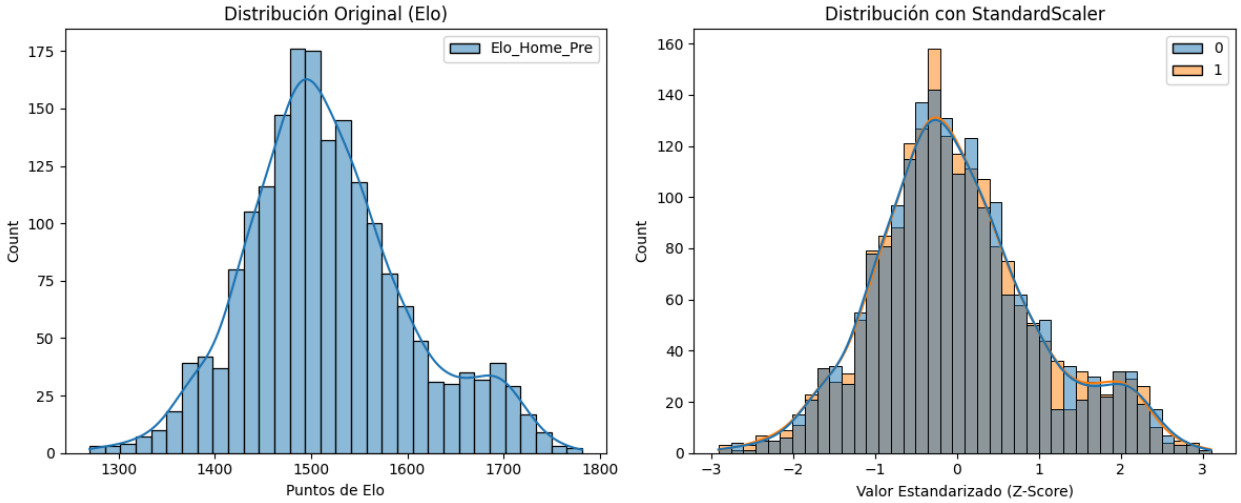
Sunderland	1479.57
Watford	1463.34
Norwich	1462.36
West Ham	1457.55
Tottenham	1444.29
Luton	1442.44
Sheffield Utd	1408.22
Ipswich	1394.32
Leicester	1391.29
Wolves	1363.20
Southampton	1346.49
Burnley	1345.86

Standard Scaler:

La estandarización de las variables numéricas, en específico, el puntaje dinámico de fuerza relativa, es una etapa crítica en nuestro proceso de ingeniería de características por las siguientes razones:

1. Estabilidad en modelos estadísticos (Regresión de Poisson): Nuestro proceso para la predicción de marcadores exactos se basa en una regresión de Poisson, lo cual estima la tasa de ocurrencia de goles (lambda) mediante una función de enlace logarítmica. Los modelos lineales generalizados y sus algoritmos de optimización matemática son más eficientes y estables cuando las variables predictoras están en una escala uniforme. Al transformar la distribución original (Elo), la cual tiene puntuaciones entre 1300 y 1800, a una escala de puntuación Z (donde la media es 0 y la desviación estándar es 1), evitamos que esta variable domine artificialmente sobre otras características de menor magnitud

numérica como los días de descanso o ventaja de localía, como se muestra en la siguiente imagen.



2. Preservación de la distribución y mitigación del sesgo: Como se observa en las gráficas, el mapeo lineal de StandardScaler conserva la geometría y el comportamiento estadístico de los datos originales, lo que nos garantiza que el comportamiento bimodal incipiente y los extremos competitivos de la liga bi se distorsionen, manteniendo el valor predictivo de la característica para futuros ensambles como con Random Forest.

Selección de características

Justificación:

En esta fase del proyecto, determinamos no utilizar de forma automatizada los métodos estadísticos tradicionales de filtrado masivo o reducción de dimensionalidad, ya que nos vamos a enfocar en 2 variables únicamente. Basamos esta decisión en los siguientes criterios:

- Ingeniería guiada por dominio: A diferencia de problemas con muchas variables, el diseño de características de nuestro proyecto parte de una selección de predictores macro y microestructurales respaldados por la literatura científica del análisis deportivo de Maher, 1982 y Galvan Bellman, 2024. Cada variable generada como fuerza neta ELO, días de descanso y ventaja por localía, posee

una causalidad lógica directa con el desempeño y la probabilidad de éxito en cada partido.

- Preservación de la interpretabilidad del negocio: Aplicar técnicas de extracción como análisis de componentes principales (PCA) nos afectaría negativamente la interpretabilidad física de nuestras variables, fusionando variables de descanso y rendimiento en ejes abstractos. Esto también nos impactaría en el proyecto ya que requerimos que los pronósticos probabilísticos sean lo más transparentes posibles y explicables para los analistas como para los usuarios finales.
- Control de multicolinealidad mediante filtro manual: Decidimos implementar un criterio basado en matrices de correlación. Identificamos que la diferencia neta de fuerza Elo es una combinación lineal directa entre Elo_Home_Pre y Elo_Awat_Pre. Para evitar redundancia en modelos sensibles a colinealidad como las regresiones lineales o logísticas, decidimos conservar las variables individuales para el modelo de Poisson y la diferencia neta como un predictor para el clasificador de Random Forest, optimizando la complejidad del modelo y el tiempo de entrenamiento sin perder información.

Ingeniería de características para datos no estructurados

Para incorporar el contexto dinámico y cualitativo del entorno deportivo, implementamos un notebook para procesamiento de lenguaje natural. A partir de artículos recopilados de fuentes informativas, se ejecutaron operaciones comunes de ingeniería de características para transformar texto en variables cuantitativas listas para el modelado.

La primera operación consiste en la mitigación de valores faltantes y la unificación de la carga informativa, utilizando una función de sustitución y concatenación, los campos categóricos sueltos de título y descripción, se fusionaron en una nueva característica llamada “text”.

```
df news["text"] = df news["title"].fillna("") + " " + df news["description"].fillna("")
```

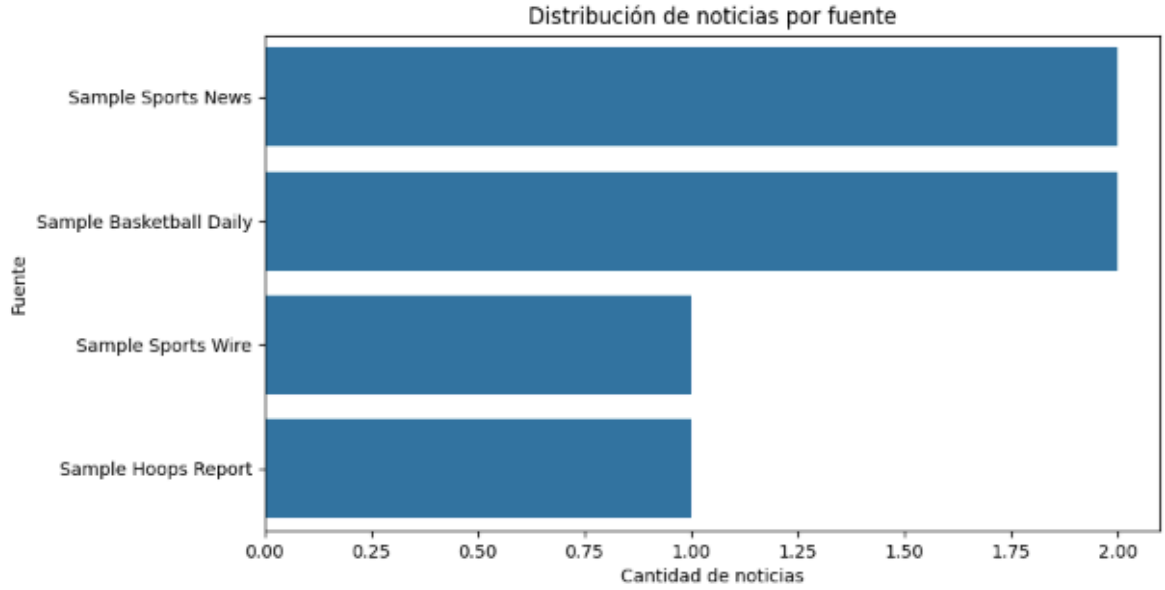
```python
categorical_cols = df_news.select_dtypes(include=["object"]).columns.tolist()

cardinality = pd.DataFrame({
    "column": categorical_cols,
    "unique_values": [df_news[col].nunique() for col in categorical_cols]
})

display(cardinality)

print("Distribución de fuentes:")
display(df_news["source"].value_counts())

sns.countplot(y="source", data=df_news, order=df_news["source"].value_counts().index)
plt.title("Distribución de noticias por fuente")
plt.xlabel("Cantidad de noticias")
plt.ylabel("Fuente")
plt.show()
```

	column	unique_values
0	title	6
1	description	6
2	source	4
3	text	6

Distribución de fuentes:

source	count
Sample Sports News	2
Sample Basketball Daily	2
Sample Sports Wire	1
Sample Hoops Report	1

dtype: int64

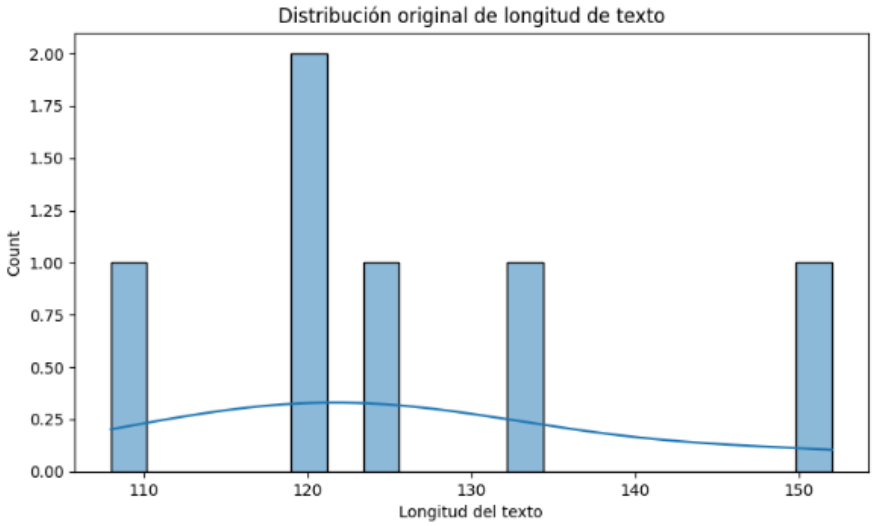


Justificación

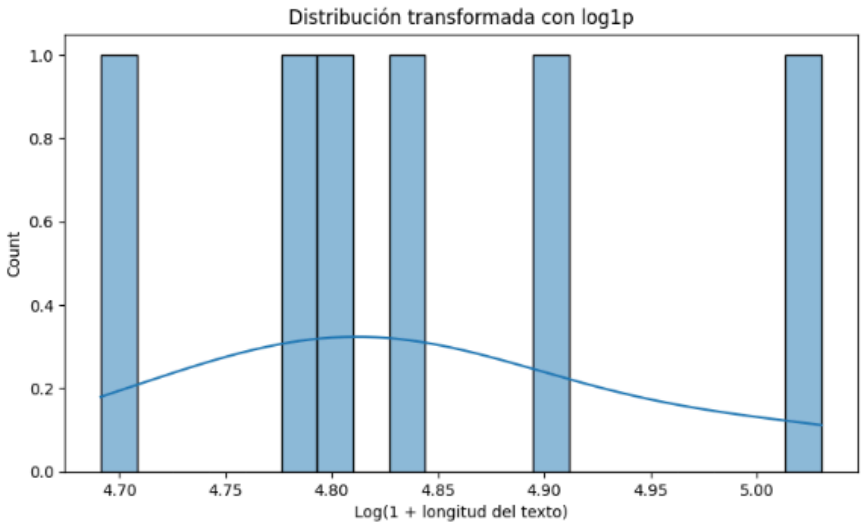
En el entorno mediático, muchas alertas críticas como jugadores que a último minuto no pueden participar en el partido, aparecen únicamente en los titulares de los textos. Esta operación nos ayuda a prevenir la pérdida de registros de valores NaN en la descripción del texto y proporciona un bloque semántico completo para los algoritmos de extracción.

A partir del texto consolidado, se estructuraron dos variables numéricas continuas para capturar la densidad del entorno del equipo o del jugador:

Longitud del texto (text_length): Variable numérica que contabiliza la cantidad total de caracteres por registro.
Transformación logarítmica (log_text_length): Aplicación de la función no lineal np.log1p sobre la longitud calculada.




```
Skewness original: 0.9796093622949777
```




```
Skewness después de log1p: 0.7217892753277613
```

Justificación

El EDA nos demostró que la distribución original de la longitud de noticias presenta un sesgo hacia la derecha debido a notas atípicas. La aplicación de la transformación log1p reduce este sesgo, estabilizando la varianza del predictor. Es indispensable esta parte para proteger al pipeline predictivo contra valores atípicos (outliers) y asegurar la estabilidad en componentes lineales.

Para que los algoritmos puedan interpretar patrones de texto, se implementó una técnica de codificación basada en la frecuencia de términos y la frecuencia inversa de los documentos (TF-IDF), utilizando la clase TfidVectorizer:

```python
tfidf = TfidfVectorizer(
    max_features=20,
    stop_words="english"
)

X_tfidf = tfidf.fit_transform(df_news["text"].fillna(""))

tfidf_terms = pd.DataFrame({
    "term": tfidf.get_feature_names_out()
})

display(tfidf_terms)
```

	term
0	according
1	analysts
2	arrives
3	betting
4	celtics
5	changes
6	coach
7	confirms
8	consecutive
9	continue
10	day
11	deal
12	defense
13	defensive
14	game
15	improved
16	injury
17	performance
18	player
19	team

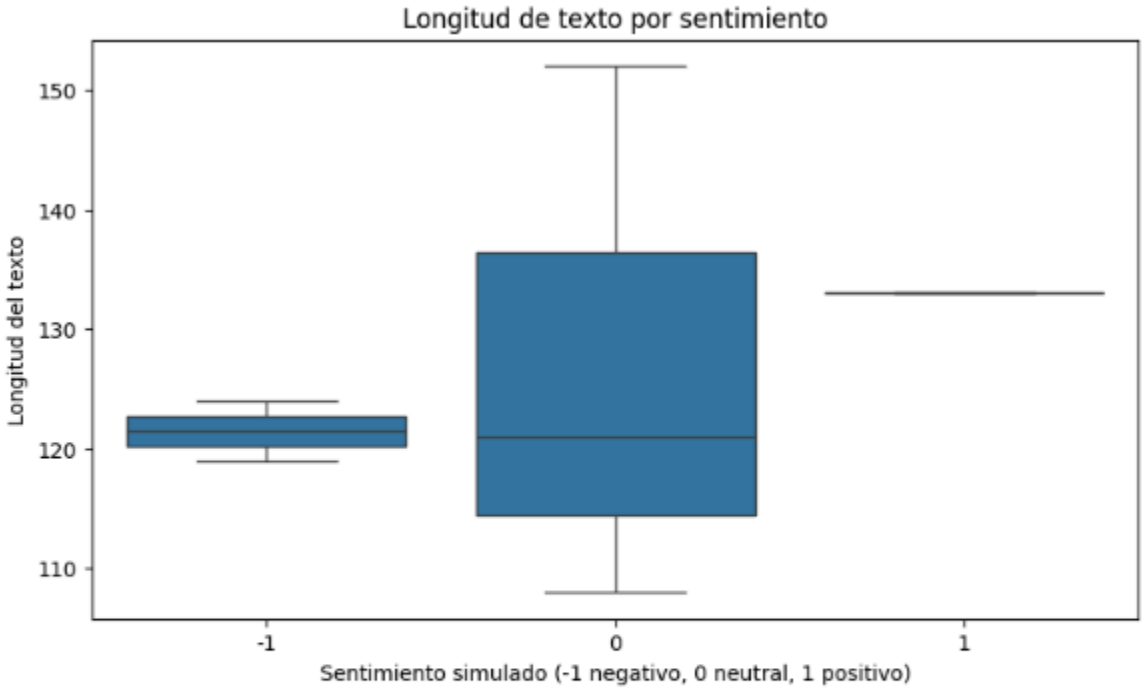
Justificación

TF-IDF penaliza los términos que aparecen con demasiada frecuencia en todo el set de datos, y le da más peso a aquellas palabras clave con alta carga semántica contextual como “injury”.

Como se puede ver en la imagen, se fijó el hiperparámetro max_features=20 y se aplicó un filtro de palabras vacías (stop_words = “english”), esto para que actúe como un filtrado manual directo que restringe la cardinalidad espacial a los 20 términos más informativos del documento, disminuyendo los requerimientos de almacenamiento, complejidad computacional y mitigando el riesgo de sobreajuste.

Finalmente el notebook de NPL culmina su pipeline con la preparación de variables para la validación del flujo de machine learning antes de su integración con los marcadores cuantitativos de fútbol y NBA:

- Puntuación de sentimiento: Es la variable categórica ordinal que codifica el estado anímico y situacional en un espacio de 3 clases:
    - -1: Contexto adverso/crisis)
    - 0: Neutral
    - 1: Contexto favorable.

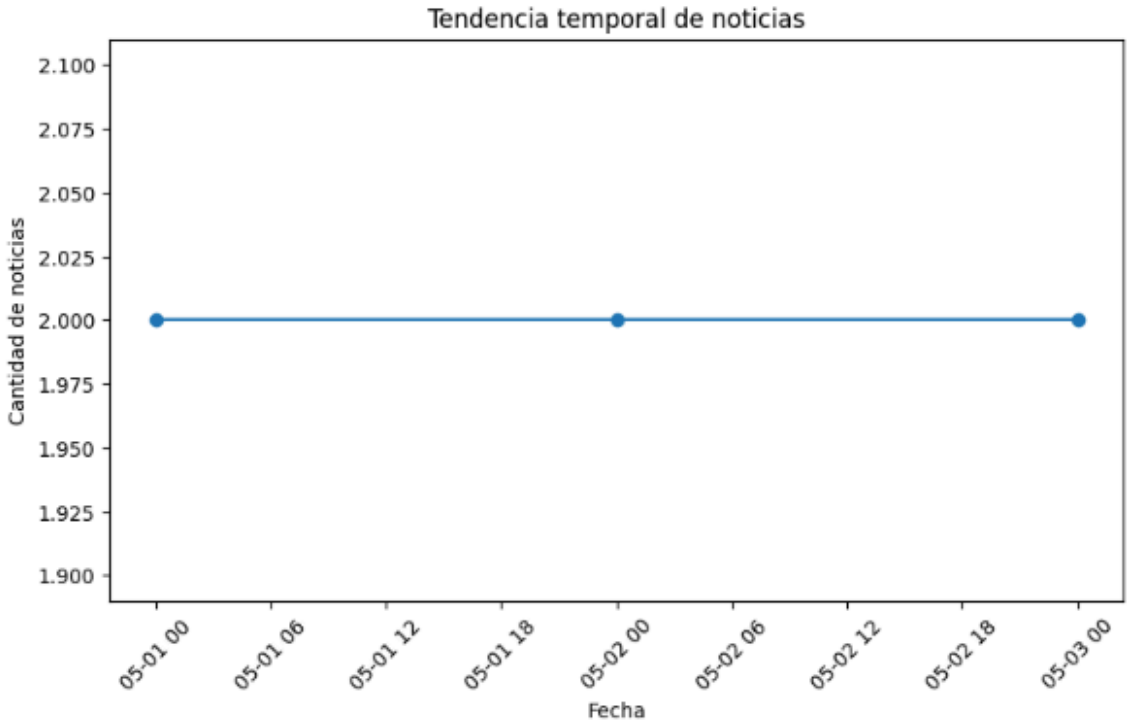


sentiment_score	count	mean	std	min	25%	50%	75%	max
-1	2.0	121.5	3.535534	119.0	120.25	121.5	122.75	124.0
0	3.0	127.0	22.605309	108.0	114.50	121.0	136.50	152.0
1	1.0	133.0	NaN	133.0	133.00	133.0	133.00	133.0

- En la etapa inicial del proyecto, esta columna se alimenta de forma automatizada mediante modelos preentrenados o APIs de procesamiento de lenguaje en la nube.
- La columna “publishedAt”, se transformó a un tipo de dato temporal homogéneo (datetime) y luego se extrajo la fecha neta en “published_date”.

```
df_news["published_date"] = df_news["publishedAt"].dt.date

news_by_date = df_news.groupby("published_date").size().reset_index(name="news_count")

display(news_by_date)

plt.plot(news_by_date["published_date"], news_by_date["news_count"], marker="o")
plt.title("Tendencia temporal de noticias")
plt.xlabel("Fecha")
plt.ylabel("Cantidad de noticias")
plt.xticks(rotation=45)
plt.show()
```

	published_date	news_count
0	2026-05-01	2
1	2026-05-02	2
2	2026-05-03	2



Justificación

La normalización de las marcas de tiempo y la extracción de la fecha limpia son obligatorias para posibilitar el cruce de datos e interconexión con las tablas estructuradas de los partidos. Esto nos asegura que el modelo final del proyecto consuma características de NLP balanceadas que representen el estado de la información disponible antes de que inicie cada partido, solucionando temas de *data leakage.*

Conclusiones de la fase

De acuerdo con el ciclo de vida de desarrollo de sistemas de aprendizaje automático (**CRISP-ML(Q)),** la fase de preparación de los datos e ingeniería de características nos ha permitido consolidar los siguientes puntos en nuestro proyecto:

- Mitigar el data drift: Al crear el algoritmo de reinicio parcial estacional (es decir, mantener el 67% del puntaje acumulado y reincorporar el 33% del estándar de 1500 puntos) obtenemos una solución basada en MLOps, que consigue balancear la memoria histórica del rendimiento de un equipo con la disrupción natural provocada por situaciones que ocurren entre temporadas (como traspaso de jugadores, lesiones, etc.).
- Estandarización y escalabilidad: El procesamiento y reestructuración paralela de las variables de NBA demuestra que la arquitectura del proceso está diseñada para ser independiente de cualquier deporte. Esto nos da mensajes positivos para expandir el producto hacia una solución que pueda ser aplicable a otros deportes, inicialmente el fútbol soccer.
- Sinergia para la siguiente fase: Al haber utilizado ingeniería de características para transformar métricas crudas en distribuciones estandarizadas, codificado el contexto espacial de los partidos y controlado la redundancia de variables, se ha obtenido un conjunto de datos óptimo para trabajar. El set de datos resultante nos permite alimentar al modelo de Poisson como al modelo de ensamble no lineal (Random Forest), asegurando una transición sólida a la siguiente fase, donde también buscaremos incluir el sistema de NLP para análisis de noticias acerca de equipos y jugadores.

Referencias

Leidwein, F. (2025). *How to Build an ELO Rating System for the Premier League in Python.* Recuperado de: https://statsultra.com/how-to-build-an-elo-rating-system-for-the-premier-league-in-python/

Amazon Web Services. (2024). Amazon Comprehend Developer Guide. https://aws.amazon.com/comprehend/

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Galván Bellman, J. C., Méndez González, L. C., Pérez Olguín, I. J. C., & Quezada Carreón, A. E. (2024). Algoritmo de Machine Learning para la Predicción de un Partido de Fútbol. *Revista Academia Journals*, 16(4).

GDELT Project. (2024). Global Database of Events, Language, and Tone. https://www.gdeltproject.org/

Google Cloud. (2024). Natural Language API Documentation. https://cloud.google.com/natural-language

Maher, M. J. (1982). Modelling association football scores. *Statistica Neerlandica*, 36(3), 109-118.

Microsoft Azure. (2024). Text Analytics Documentation. https://azure.microsoft.com/en-us/products/ai-services/text-analytics/

NewsAPI. (2024). News API Documentation. https://newsapi.org/

Visengeriyeva, L., Kammer, A., Bär, I., Kniesz, A., y Plöd, M. (2023). *CRISP-ML(Q). The ML Lifecycle Process*. MLOps. INNOQ. https://ml-ops.org/content/crisp-ml

La frase de Michael Jordan que explica a la perfección la importancia del trabajo en equipo - Elige Educar. (2017, 22 septiembre). Elige Educar. https://eligeeducar.cl/ideas-para-el-aula/la-frase-de-michael-jordan-que-explica-a-la-perfeccion-la-importancia-del-trabajo-en-equipo/